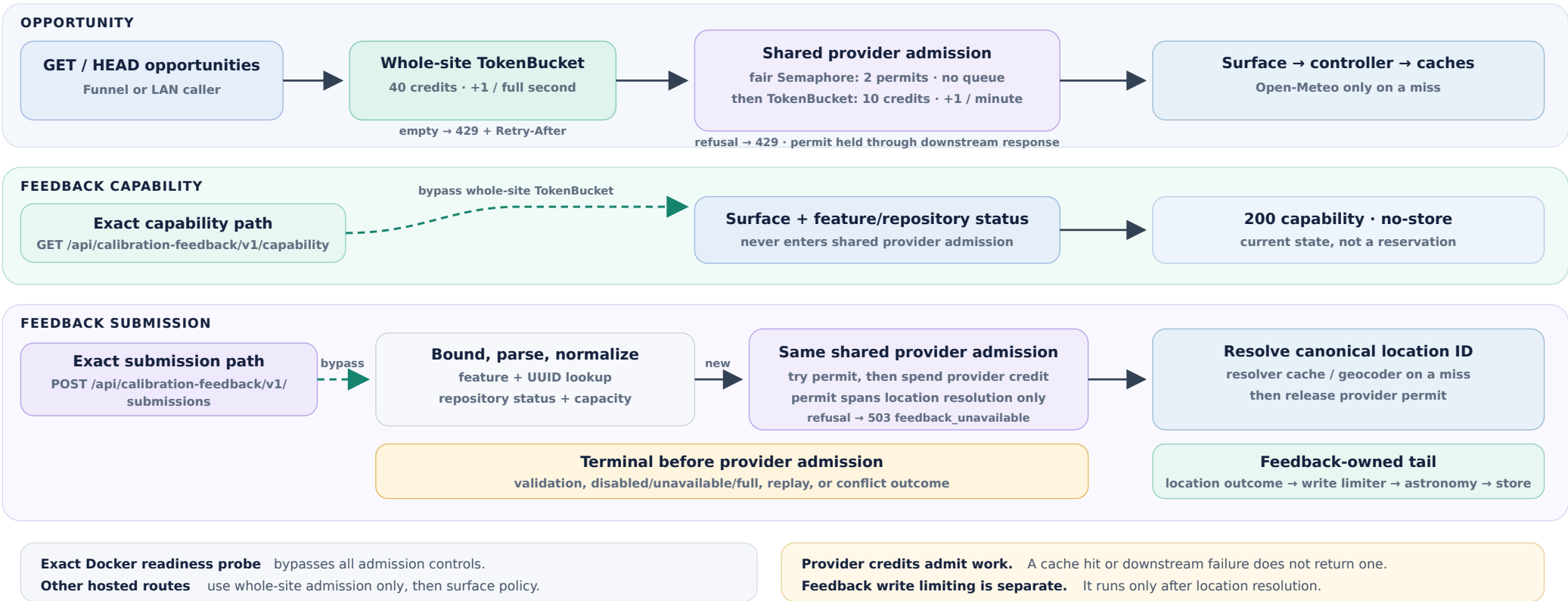


Admission by route

Feedback skips whole-site admission; new-report location resolution still shares provider admission with opportunities.

Three request paths · one process-wide provider guard

SHARED BY ALL VISITORS



Route matrix

Opportunity	whole-site + shared provider admission
Capability	neither admission control
New feedback	provider admission after cheap checks
Other routes	whole-site only
Docker probe	neither admission control

Refusal contract

- Whole-site exhaustion: 429 + computed Retry-After.
 - Opportunity permit refusal: 429 + Retry-After: 1.
 - Opportunity provider-token refusal: 429 + retry hint.
 - Feedback shared-gate refusal: 503 feedback_unavailable.
 - Feedback's later write limiter can return its own 429.
- Public feedback errors never reveal which shared guard refused work.

Shared provider bound

- One fair two-permit Semaphore is process-wide.
- One 10-credit TokenBucket refills once per minute.
- Opportunity holds a permit through its full filter chain.
- Feedback holds a permit only while resolving location.

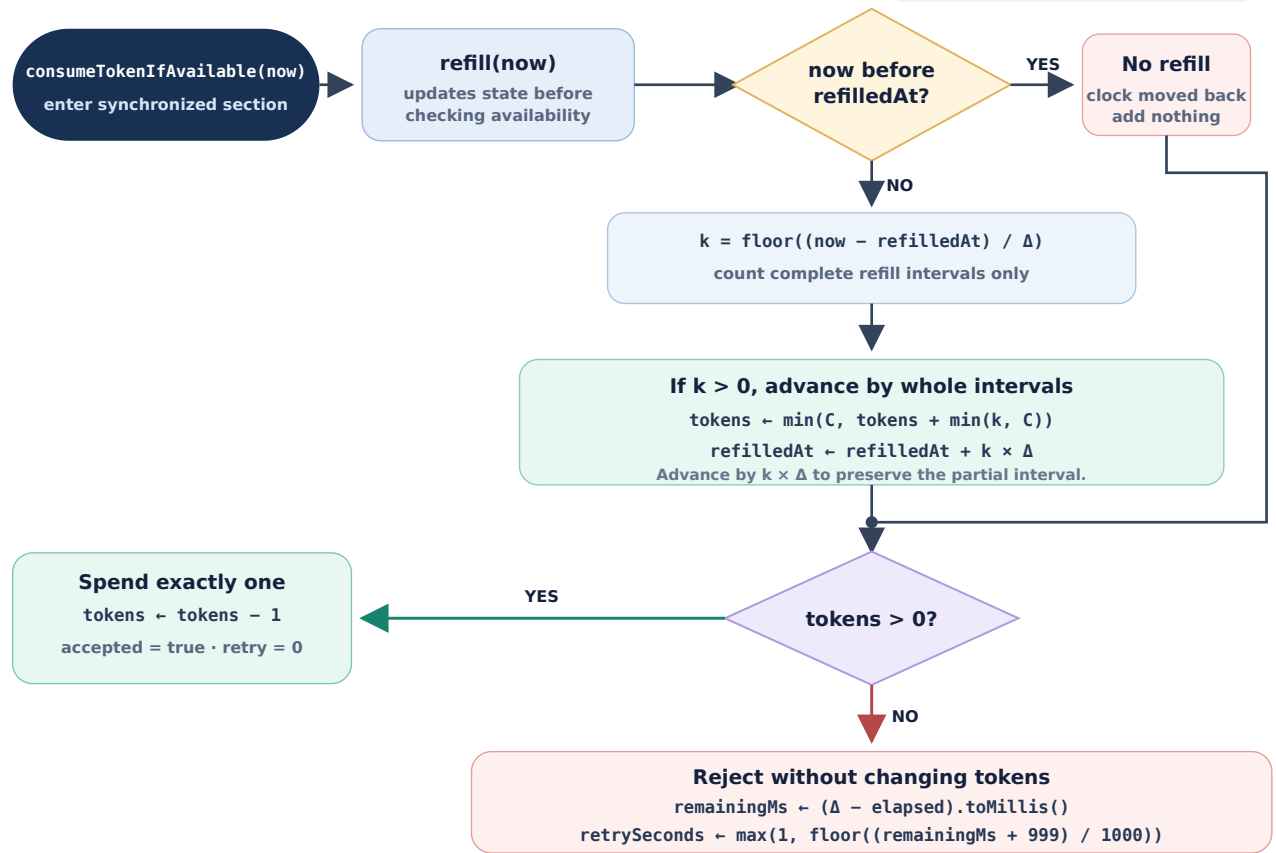
Uninterrupted 24-hour proof

1,450 shared admissions × at most 4 attempts = 5,800 attempts.

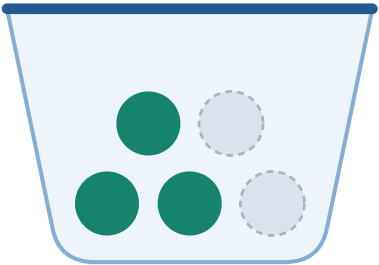
Inside one TokenBucket

A synchronized call refills complete intervals, spends one admission credit, or returns a bounded retry hint.

What synchronized consumeTokenIfAvailable(now) does LAZY REFILL · NO BACKGROUND TIMER



+1 after each complete Δ State carried between calls



$0 \leq \text{tokens} \leq \text{capacity } C$
illustrative slots; capacity is configurable

Four fields

capacity C	maximum stored credits
refill interval Δ	time required for one credit
tokens	current stored admission credits
refilledAt	anchor; advances even when capacity-capped

Whole-site instance

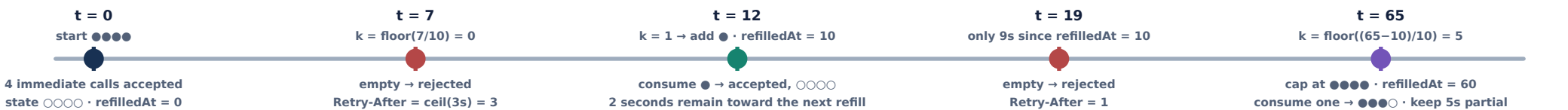
$C = 40 \cdot \Delta = 1s$
Shared except exact Docker
and feedback API paths.

Provider instance

$C = 10 \cdot \Delta = 60s$
Shared by opportunities and
feedback location resolution.

The feedback write limiter is separate; it is not this TokenBucket.

Worked timeline · illustrative $C = 4, \Delta = 10$ seconds



Clock and concurrency safeguards

A backward clock never mints credits; an empty bucket then uses zero elapsed time. Synchronization prevents simultaneous callers from spending the same credit. Downstream failure does not return a spent token; only timed refill or a process restart restores it.